

 Week 1 > Fnd 01 · Solution

Foundations 01 — Positron and the R Environment

STAT/DATA 1810 · Week 1 Wednesday · SOLUTION

AUTHOR

Instructor key

Name: Instructor key · **Date:** _____

Timing & labels

- **Budget:** panes + session 12' · running code 12' · objects 16' · functions/packages/help 20' · errors 12' · save & render 8' = **80'**
- **Checkpoint:** end of §2 at the **40' mark** — if behind, cut YT6 and YT7c
- **LM Learning Moment** — instructor demonstrates, students follow
- **YT Your Turn** — students do it themselves
- Say both names out loud the first time each appears — students otherwise guess

0 · Getting set up and what you are looking at 12 min

Setup checklist

Prep page

- Big-font, step-by-step version for the projector: [Prep — Getting Set Up](#) (also covers the Session tab and LM/YT)

Teaching note — setup

- The eight setup steps should be done **before** class: install R → install Positron → download the **.qmd** → make **Desktop/R/{Notes,Labs}** and **Desktop/Data** → move the file into **R/Notes** → open Positron → **File** → **Open Folder...** on **Desktop/R** → open the file from the sidebar → fill in name and date
- Budget **3 minutes** at the start to rescue whoever isn't there yet
- Most common failures:
 - the file is still in **Downloads**
 - they opened the *file* rather than the *folder*, so the Explorer sidebar is empty
- Fix for both: **File** → **Open Folder...** → **Desktop/R**
- Make them fill in name and date — first thing they type into a **.qmd**, and how you tell whose render is whose

The panes 6 min

Teaching note — panes

- Put the annotated screenshot (`images/positron_panes.png`) on the projector
- Have students physically point at each numbered pane — the checklist **is** the activity
- Use Positron’s own names consistently: **Editor**, **Console**, **Variables**, **Plots**, **Explorer**
- Two traps:
 - **CONSOLE** and **TERMINAL** tabs sit side by side — the Terminal shows a `%` prompt, not `>`
 - Variables/Plots live inside the **SESSION** panel on the right — if closed, the right side looks blank
- Terminal / Viewer / Connections get one sentence each — resist the tour becoming the lesson

Answers

- **YT1** Variables
- **YT2** the Editor (this `.qmd`)

The Session 6 min

Teaching note — getting to the Session tab

- Point at the top-right interpreter label (`R 4.x.x` , arrow 6 on the screenshot)
- Click it once so they see the menu: running session, Restart, Shutdown, other interpreters — then press Escape
- Point at the **SESSION** panel beneath it: *“that label is the session; this panel is what the session remembers”*
- Say the three facts — **memory / restartable / replaceable** — and move on
- Payoff comes in §4 LM11 when they restart for real
- Show that Variables is empty *now* — the “before” picture makes the restart in §4 land

Q1 model answer

- **Q1** The Console runs code once and forgets it
- The Editor holds code you keep, so you can re-run everything tomorrow

1 · Running code and code chunks 12 min

Teaching notes — LM1-LM3

- **LM1** The up-arrow trick saves students dozens of retypes this term — worth the 20 seconds
- **LM2 Anatomy of a chunk:**
 - put the cursor on the ```{r}` line and name its parts out loud: fence, `{r}` = “this is R”, fence
 - click the play button so they see output land **both** under the chunk and in the Console
 - the sentence to leave them with: *“a chunk is a saved bundle of Console lines”*
 - students create their own chunks starting Monday (Foundations 02) — today they only run and edit ours
- **LM3** One sentence on `#|` options is enough — they’ll write `#| label:` and `#| eval: false` themselves in ETV cycles

```
# LM2  
2 + 2
```

```
[1] 4
```

```
3 * 7
```

```
[1] 21
```

```
10 / 4
```

```
[1] 2.5
```

```
# YT3  
5 - 4
```

```
[1] 1
```

```
100 / 8
```

```
[1] 12.5
```

```
2^10
```

```
[1] 1024
```

Expected – YT3

- **YT3** `1 · 12.5 · 1024`
- **Q2** **both** (output appears under the chunk *and* in the Console)

2 · Objects and assignment 16 min

```
# LM4  
cups <- 3  
cups
```

```
[1] 3
```

```
# LM5 – prediction: 13.5  
price <- 4.50  
total <- cups * price  
total
```

```
[1] 13.5
```

```
# LM6
cups <- 10
total
```

```
[1] 13.5
```

Q4 model answer — the most important idea of the day

- Q4 `total` was computed *at the moment* the `<-` line ran, using `cups = 3`
- Changing `cups` later doesn't re-run old lines — **R objects store values, not formulas**
- Spend the full minutes here
- Spreadsheet contrast lands well: Excel cells recompute; R objects don't

```
# YT4 — any sleep value; e.g.
hours <- 7
minutes <- hours * 60
minutes
```

```
[1] 420
```

```
hours <- 8
minutes          # still 420 — same lesson as Q4, now self-discovered
```

```
[1] 420
```

YT4 (d) answer

- YT4 `minutes` does **not** change — Q4 again, but the student built it, so it sticks
- Have two students say their prediction and result aloud
- Then point at Variables: `cups`, `price`, `total`, `hours`, `minutes` — five objects, all session memory

3 · Functions, packages, help 20 min

```
# LM7
mean(mtcars$mpg)
```

```
[1] 20.09062
```

```
nrow(mtcars)
```

```
[1] 32
```

Expected — LM7

- LM7 20.09062 · 32

```
# YT5
max(mtcars$hp)
```

```
[1] 335
```

```
min(mtcars$mpg)
```

```
[1] 10.4
```

```
ncol(mtcars)
```

```
[1] 11
```

```
round(mean(mtcars$mpg), 1)
```

```
[1] 20.1
```

Expected — YT5

- YT5 335 (the Maserati Bora) · 10.4 · 11 · 20.1
- Part (d) is the first *nested* call — say “inside out” once; don’t dwell

Q5 · YT6 · Q6 answers

- Q5 `na.rm = TRUE` tells `mean()` to remove `NA` values before computing; the default `FALSE` returns `NA` if any value is missing
 - plant the flag: this argument returns in Foundations 03 and every ETV cycle
- YT6 `digits`, default `0` — the purpose is reading a Usage line, not the fact itself
- Q6 install runs **once per computer**; library runs **once every session**
 - analogy that sticks: install = buying the drill; library = taking it out of the garage

```
library(tidyverse)
```

4 · When things break 12 min

```
meen(mtcars$mpg)
#> Error in meen(mtcars$mpg) : could not find function "meen"
```

```
# LM10 fixed
mean(mtcars$mpg)
```

```
[1] 20.09062
```

Q7 model answer

- **Q7** “could not find function” means the *name* doesn’t exist — a typo, or a package not loaded
- Name the two most common error families now:
 - **could not find function** → typo / `library()` missing
 - **object not found** → name typo / never created
- Students who can *classify* an error stop fearing errors

YT7 answers — three error families

- **YT7a** `mtcars$mpg` → returns **NULL**; `mean` warns “argument is not numeric or logical: returning NA”
 - family: **column-name typo** — `$` on a missing column gives **NULL**, not an error (the sneakiest of the three)
 - fix: `mpg`
- **YT7b** `Mtcars` → **Error: object 'Mtcars' not found**
 - family: **object not found** — R is case-sensitive
 - fix: `mtcars`
- **YT7c** missing `)` → Console shows a **+** prompt waiting for more; in a chunk, “unexpected end of input”
 - family: **unbalanced parentheses**
 - fix: add `)` — and show them **Escape** in the Console to get out of a **+** prompt

```
mean(mtcars$mpg)
```

```
[1] 20.09062
```

```
mean(mtcars$mpg)
```

```
[1] 20.09062
```

```
mean(mtcars$mpg)
```

```
[1] 20.09062
```

LM11 — restart for real

- **LM11** Everyone restarts → Variables empties
- **Q8** `cups` gone · code still here · re-running §2 takes ~10 seconds
- That 10-second recovery *is* the argument for putting code in the file

5 · Save & render 8 min

Exit model answer

- `cups` lives in Variables → gone after restart

- this `.qmd` lives in the Editor → survives
- the code that *recreates* `cups` survives — the entire argument for reproducible documents, made on day one without ever saying “CLO 9”